

Résumé du projet - Overlook Hotel

Overlook Hotel est une application web de gestion hôtelière réalisée en équipe dans le cadre de la formation Concepteur Développeur d'Applications. Le projet répond à un double besoin : simplifier le parcours de réservation des clients et centraliser les activités nécessaires au fonctionnement quotidien d'un établissement hôtelier.

L'application propose plusieurs espaces adaptés aux rôles des utilisateurs. Les visiteurs peuvent consulter les chambres, rechercher les disponibilités et découvrir les services de l'hôtel. Une fois authentifiés, les clients peuvent réserver une chambre, suivre leurs réservations, consulter leurs informations personnelles, leurs avis et leur programme de fidélité. Les employés disposent d'outils pour consulter leur planning, suivre leur temps de travail et déposer des demandes de congé. Les responsables et administrateurs peuvent gérer les chambres, les réservations, les utilisateurs, les rôles et les demandes du personnel.

Au cours du projet, j'ai principalement travaillé sur le parcours client et la gestion des données. J'ai participé à la réalisation de l'espace client, à la recherche des chambres disponibles, à la réservation et à l'affichage des avis. J'ai également développé et amélioré la gestion des demandes de congé, l'initialisation des utilisateurs et des chambres, ainsi que l'association cohérente des comptes, des clients, des employés et de leurs rôles. J'ai enfin contribué à la sécurisation de l'application, aux corrections fonctionnelles et à la stabilisation des échanges entre les interfaces, les services et la base de données.

Le serveur est développé en Java avec Spring Boot. Les interfaces utilisent Thymeleaf, JavaScript, HTML et CSS. Les données métier sont enregistrées dans PostgreSQL avec Spring Data JPA, tandis que Flyway versionne les évolutions du schéma. L'application suit une architecture en couches séparant les contrôleurs, les services, les repositories, les entités et les vues. MapStruct facilite les conversions entre les objets métier et les objets échangés par l'API.

La sécurité repose sur Spring Security, une gestion des autorisations par rôles, le hachage des mots de passe et des jetons JWT pour les routes protégées. Redis conserve temporairement les jetons invalidés lors de la déconnexion et met en cache certaines consultations. Les routes REST sont documentées avec Swagger/OpenAPI afin de faciliter leur compréhension et leur vérification.

Le projet a été mené à partir d'un cahier des charges, de personas, de user stories, de wireframes, de maquettes Figma et de modèles de données. Le travail collaboratif s'est appuyé sur GitHub et une organisation par branches. J'ai particulièrement contribué à l'industrialisation du projet : conteneurisation avec Docker, environnements distincts de développement et de recette, migrations reproductibles, workflows GitHub Actions et contrôles de qualité avec Spotless, PMD, SpotBugs/FindSecBugs et JaCoCo.

Des tests unitaires et des tests d'intégration en environnement Docker vérifient les parcours critiques : authentification, déconnexion, autorisations, consultation des chambres, réservations, fidélité, plannings et congés. Cette démarche permet de comparer le comportement attendu au résultat obtenu et de préparer une livraison plus fiable.

Ce projet m'a permis de mettre en pratique l'ensemble du cycle de réalisation d'une application professionnelle : analyse du besoin, conception, développement full stack, sécurisation, persistance des données, travail en équipe, tests, intégration continue et préparation au déploiement.